

RUGGINE

Applicazione di Chat Client/Server

con gruppi, log di sistema e portabilità multi-piattaforma

343432 - Omar Simone
349544 - Giulia Cristina Varga

Agenda

01 Obiettivi e requisiti del progetto

02 Architettura del sistema

03 Funzionalità principali

04 Tecnologie e linguaggio: Rust

05 Prestazioni e log di sistema

06 Portabilità multi-piattaforma

07 Dimensione eseguibile e risultati

Obiettivi e Requisiti

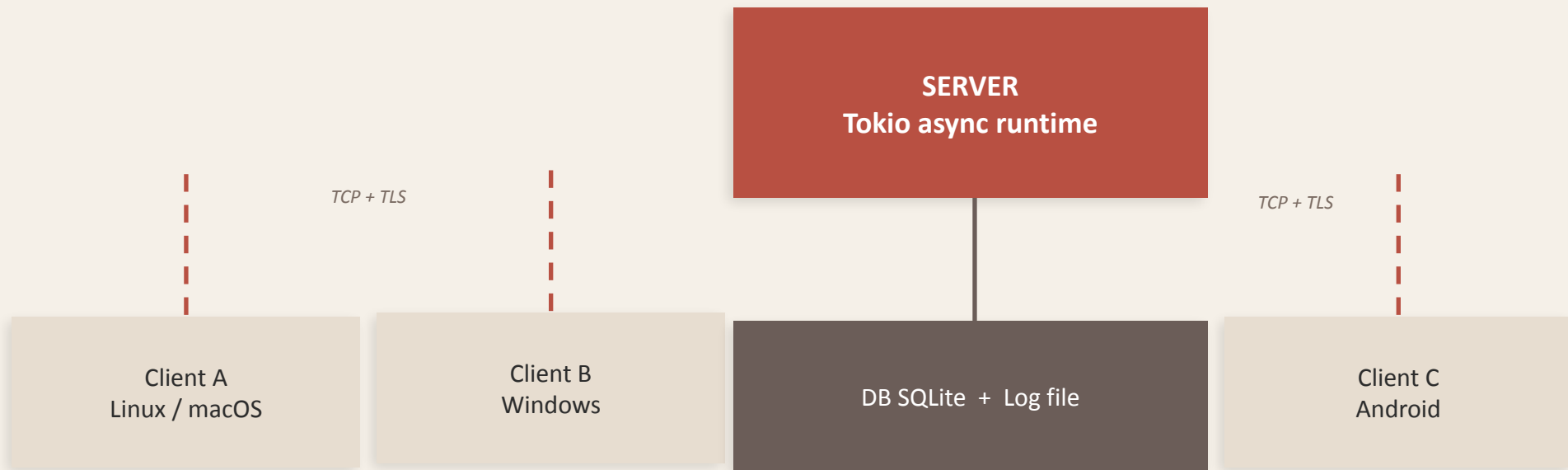
Requisiti Funzionali

- Chat testuale client/server
- Iscrizione al primo avvio (al server)
- Creazione e gestione di gruppi
- Ingresso ai gruppi solo su invito
- Scambio messaggi in tempo reale

Requisiti Non Funzionali

- Portabilità: ≥ 2 piattaforme
- Basso consumo CPU e RAM
- Eseguitibile compatto
- Log CPU ogni 2 minuti
- Performance misurate e documentate

Architettura del Sistema



Protocollo: JSON su TCP – autenticazione token JWT

Funzionalità Principali

Registrazione

Al primo avvio il client invia una richiesta di iscrizione al server. Le credenziali vengono salvate in modo sicuro con hashing (bcrypt).

Messaggistica

Messaggi testuali in tempo reale tra utenti. Il server smista i messaggi in modo asincrono tramite canali Tokio.

Gruppi su invito

Un utente crea un gruppo e invita altri utenti. L'accesso è possibile solo tramite invito esplicito del creatore del gruppo.

Persistenza

Tutti i messaggi e le informazioni di gruppo sono persistiti in un database SQLite locale al server, garantendo robustezza.

Tecnologie: perché Rust?



Linguaggio scelto:

Rust 1.78

Stack: Tokio · Serde · SQLite · TLS



Memory safety

Nessun garbage collector: zero pause impreviste e consumo RAM predicibile.



Performance

Binari nativi con overhead minimo, ideali per applicazioni di rete ad alta concorrenza.



Concorrenza sicura

Il borrow checker elimina race conditions a compile time, senza overhead runtime.



Portabilità

Cross-compilation nativa: un solo codebase per Windows, Linux, macOS, Android.



Cargo

Ecosistema moderno: dipendenze, test e build in un unico strumento.

Prestazioni e Log di Sistema

< 1%

CPU idle
(server)

~8 MB

RAM in uso
(idle)

2 min

Intervallo
log CPU

Formato del file di log

```
[2025-03-14 10:00:00] CPU: 0.42%  Threads: 12  Uptime: 00:02:00
[2025-03-14 10:02:00] CPU: 0.61%  Threads: 12  Uptime: 00:04:00
[2025-03-14 10:04:00] CPU: 0.38%  Threads: 14  Uptime: 00:06:00
```

Portabilità e Dimensione Eseguitibile

Piattaforma	Architettura	Dim. eseguibile	Testato
Linux	x86_64	~3.2 MB	✓
Windows	x86_64	~3.8 MB	✓
macOS	ARM64	~3.5 MB	✓
Android	aarch64	~4.1 MB	✓

Cross-compilation

```
cargo build --target  
aarch64-linux-android
```

Un solo codebase,
multi-target con Cargo

Ottimizzazione dimensione eseguibile

Compilazione con `opt-level = "z"` · LTO abilitato · `strip = true` → riduzione fino al 40% rispetto al debug build

Conclusioni

- ✓ Sistema client/server funzionante con chat testuale e gruppi su invito
- ✓ Implementato interamente in Rust: sicurezza, performance e portabilità
- ✓ Portabile su Linux, Windows, macOS e Android con un unico codebase
- ✓ Log CPU ogni 2 minuti: consumo medio < 1% idle
- ✓ Eseguitibile compatto: ~3–4 MB con LTO e strip abilitati